

## Code-Layout, Readability and Reusability

|                      |              |
|----------------------|--------------|
| <b>Date</b>          | 12 July 2026 |
| <b>Team ID</b>       |              |
| <b>Project Name</b>  | Smart Lender |
| <b>Maximum Marks</b> | 5 Marks      |

### Code Layout Checklist:

| S.No | Code Quality Parameter   | Description                                                                                                      | Followed (Yes / No / Partial) | Remarks                                                                                                                |
|------|--------------------------|------------------------------------------------------------------------------------------------------------------|-------------------------------|------------------------------------------------------------------------------------------------------------------------|
| 1    | Consistent Indentation   | Uniform spacing and indentation are maintained throughout the source code for better readability.                | Yes                           | Uses standard Python indentation (4 spaces).                                                                           |
| 2    | Proper File Structure    | Files and folders are logically organized into templates, static files, datasets, models, and backend scripts.   | Yes                           | Modular project directory with separate folders.                                                                       |
| 3    | Meaningful Variable Name | Variable names clearly describe their purpose and improve code readability.                                      | Yes                           | Descriptive names such as <code>wellbeing_score</code> , <code>user_data</code> , and <code>prediction_result</code> . |
| 4    | Function / Method Names  | Functions are named according to their responsibilities and follow Python naming conventions.                    | Yes                           | Functions such as <code>load_model()</code> , <code>predict_wellbeing()</code> , and <code>preprocess_data()</code> .  |
| 5    | Code Comments            | Important sections of the code contain comments explaining the logic and workflow.                               | Yes                           | Comments are included for preprocessing, prediction, and visualization modules.                                        |
| 6    | Modular Design           | The application is divided into reusable modules for preprocessing, prediction, visualization, and Flask routes. | Yes                           | Easy to maintain and extend.                                                                                           |
| 7    | No Redundant Code        | Duplicate or unnecessary code has been removed to improve maintainability.                                       | Yes                           | Common logic is reused through functions.                                                                              |
| 8    | Error Handling           | Input validation and exception handling are implemented to avoid application crashes.                            | Yes                           | Handles invalid inputs, missing files, and prediction errors.                                                          |

### Reusable Components / Modules:

| S.No | Component / Module Name   | Language / Technology | Where Reused            | Reusability Level (High / Medium / Low) |
|------|---------------------------|-----------------------|-------------------------|-----------------------------------------|
| 1    | Data Preprocessing Module | Python, Pandas        | Training and Prediction | High                                    |

|   |                        |                     |                                  |        |
|---|------------------------|---------------------|----------------------------------|--------|
| 2 | Machine Learning Model | Scikit-learn        | Prediction Module                | High   |
| 3 | Flask Routes           | Flask               | User Input and Result Pages      | High   |
| 4 | Visualization Module   | Matplotlib, Seaborn | Dashboard and Prediction Results | Medium |
| 5 | HTML Templates         | HTML5, Bootstrap    | Multiple Web Pages               | High   |

Overall Code Quality Assessment:

| Aspect                   | Rating (1-5) | Comments                                                                                                                                     |
|--------------------------|--------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| Code Layout & Structure  | 5            | The project follows a modular architecture with separate folders for templates, static resources, datasets, and machine learning components. |
| Readability              | 5            | Meaningful variable names, proper indentation, and clear function names improve readability.                                                 |
| Reusability              | 5            | Functions and modules are reusable for training, prediction, visualization, and future enhancements.                                         |
| Documentation / Comments | 4            | Important functions and processing steps are documented with comments. Additional API documentation can be added in future versions.         |
| Overall Score            | 4.8 / 5      | The application is well-structured, maintainable, reusable, and follows good coding practices suitable for an AI/ML Flask project.           |